

CSCE 2014 - Laboratory Assignment 8

The purpose of this lab is to introduce students to the **stack** abstract data type, and to see how stacks can be implemented using linked list techniques. We will also create a simple program to test the **stack interface** (ie. the public methods).

1. Introduction to Stacks

The stack abstract data type is used to store and access data in a LIFO (last in first out) manner. For example if we store 3 values A, B, and C in a stack. Then when we retrieve the data, we will get C, B, and A. The 'C' was the last value stored, so it will be the first value retrieved.

Stacks have found a number of uses in computer systems. Perhaps the most important use of stacks is in operating systems, where they are used to coordinate execution of instructions after servicing interrupts. Stacks are also used to implement recursion when a program is running.

Five basic operations can be performed on a stack:

Pop: To retrieve and remove the top most item from the stack

Push: To push an item on to the top of the stack

Top: To retrieve the top most item on the stack

IsEmpty: Check whether the stack is empty

IsFull: Check whether the stack is full

2. The Stack Class

Consider the C++ class declaration below for a Stack class. Notice that we have declared a SNode class with two public data fields. This is a somewhat lazy way to implement the SNode class, but we are only intending to use this class within the Stack class, so "real" users will not be using SNodes.

```
//-----  
// Purpose:    Header file for the Stack class.  
// Author:     John Gauch  
//-----  
#include <iostream>  
using namespace std;  
  
// Stack data node definition  
class SNode  
{  
    public:  
    int Number;  
    SNode *Next;  
};  
  
class Stack  
{  
    public:  
        // constructor functions
```

```

        Stack();
        ~Stack();

        // general stack operations
        void Push(int Number);
        void Pop(int &Number);
        void Top(int &Number);
        bool IsFull();
        bool IsEmpty();
        void Print();

    private:
        SNode *Head;
        int Length;
};

```

3. Stack Implementation

Consider the partial implementation of the Stack class below. Notice that we have left several functions empty on purpose. We will fill these in later on.

```

//-----
// Purpose:      Implementation of Stack class.
// Author:       John Gauch
//-----
#include "stack.h"

//-----
// Constructor function.
//-----
Stack::Stack()
{
    Head = NULL;
    Length = 0;
}

//-----
// Destructor function.
//-----
Stack::~~Stack()
{
}

//-----
// This checks to see if stack is full.
//-----
bool Stack::IsFull()
{
    return (false);
}

//-----
// This checks to see if stack is empty.
//-----
bool Stack::IsEmpty()
{
}

```

```

    return (Length == 0);
}

//-----
// This routine pushes data into the stack.
//-----
void Stack::Push(int Number)
{
    // Check for full stack
    if (IsFull())
        return;

    // Allocate space for data
    SNode *Temp = new SNode;
    if (Temp == NULL)
        return;

    // Insert data at head of list
    Temp->Number = Number;
    Temp->Next = Head;
    Head = Temp;
    Length++;
}

//-----
// This routine pops data from the stack.
//-----
void Stack::Pop(int &Number)
{
    // Check for empty stack
    if (IsEmpty())
        return;

    // Extract information from node
    Number = Head->Number;

    // Pop item from linked list
    SNode *Temp = Head;
    Head = Head->Next;
    delete Temp;
    Length--;
}

//-----
// This routine returns the data from the front of the stack.
//-----
void Stack::Top(int &Number)
{
    // ADD HERE
}

//-----
// This routine prints all records in the stack.
//-----
void Stack::Print()
{
    // ADD HERE
}

```

```

}

//-----
// Main program.
//-----
int main()
{
    // ADD HERE
}

```

4. Testing the Stack Class

Copy and paste the class declaration into a file called "stack.h" in your IDE. Then copy and paste the class implementation into a file called "stack.cpp". When you compile and run this code, it should do nothing.

Now extend the main function, to create a Stack object called "stack" and then call the Push and Pop methods to store and retrieve the numbers 3 1 4 1 5 9. To see what happens, print each value as it is pushed on the stack and when it is popped off the stack.

Cut and paste your new main function below:

```

Stack stack;
array<int, 6> numbers = {3, 1, 4, 1, 5, 9};
for (int num : numbers) {
    stack.Push(num);
    cout << "Pushed: " << num << endl;
}
cout << endl;
for (int i = 0; i < numbers.size(); i++) {
    int num;
    stack.Pop(num);
    cout << "Popped: " << num << endl;
}

```

Cut and paste your program output below:

```
Pushed: 3
Pushed: 1
Pushed: 4
Pushed: 1
Pushed: 5
Pushed: 9
```

```
Popped: 9
Popped: 5
Popped: 1
Popped: 4
Popped: 1
Popped: 3
```

5. Implementing Top and Print

You will notice that the Top and Print methods are not implemented for the stack class. As noted above, the Top method is supposed to return the value of the top element on the stack, but it is NOT supposed to remove this value. Use this information to implement Top and extend your main function to call top at different times in your Push, Pop sequence.

Cut and paste your Top method below:

```
void Stack::Top(int &Number)
{
    // Check for empty stack
    if (IsEmpty())
        return;

    Number = Head->Number;
}
```

Now you can implement the Print method. To do this you must "walk the linked list" to access each of the integer values. By this time, this process should look pretty familiar. When you have finished implementing Print, call this function several times between your Push and Pop calls to get a quick view of the stack as your program is running.

Cut and paste your Print method below:

```
void Stack::Print()
{
    cout << "Stack contents: ";
    if (IsEmpty())
    {
        cout << "(empty)";
    }
    else
    {
        SNode *Temp = Head;    // Preserve head pointer
        while (Temp != NULL)
        {
            cout << "[" << Temp->Number << " ] ";
            Temp = Temp->Next;
        }
    }
    cout << endl;
}
```

Cut and paste your main function below:

```
//-----
// Main program.
//-----
#include "Stack.h"
using namespace std;

int main()
{
    Stack stack;
    array<int, 6> numbers = {3, 1, 4, 1, 5, 9};
    for (int num : numbers) {
        stack.Push(num);
        //cout << "Pushed: " << num << endl;
        stack.Print();
    }
    cout << endl;
    for (int i = 0; i < numbers.size(); i++) {
        int num;
        stack.Pop(num);
        //cout << "Popped: " << num << endl;
        stack.Print();
    }
}
```

Finally, cut and paste your program output below:

```
Pushed: 3
Stack contents: [3]
Pushed: 1
Stack contents: [1] [3]
Pushed: 4
Stack contents: [4] [1] [3]
Pushed: 1
Stack contents: [1] [4] [1] [3]
Pushed: 5
Stack contents: [5] [1] [4] [1] [3]
Pushed: 9
Stack contents: [9] [5] [1] [4] [1] [3]

Popped: 9
Stack contents: [5] [1] [4] [1] [3]
Popped: 5
Stack contents: [1] [4] [1] [3]
Popped: 1
Stack contents: [4] [1] [3]
Popped: 4
Stack contents: [1] [3]
Popped: 1
Stack contents: [3]
Popped: 3
Stack contents: (empty)
```

6. Submit Work

When you have completed all of the steps to this lab:

- At the bottom of the page enter your name and UAID and click "Submit Lab".
- Print the resulting lab report web page to a PDF file.
- Login to Blackboard and go to the labs section.
- Upload your PDF file for grading and feedback.

Enter your FIRST name:

Enter your LAST name:

Enter your UAID number:

Submit Lab